Sistema de Gestión de Inventario Target Florida

Entrega 2 — Modelo Relacional Normalizado (3FN) y Dependencias Funcionales

Jose Daniel Cordoba Cabal — A00407856
Daniel Montenegro — A00409572
Sebastian Mejia — A00410557
Juan Felipe Castro — A00410561
Santiago Sierra Vasquez — A00410695

Universidad Icesi — Sistemas Intensivos de Datos — Monica Maria Rojas Rincon

1. Alcance

Este documento presenta el modelo relacional derivado del Modelo Entidad-Asociación corregido, construido en Oracle SQL Developer Data Modeler para Oracle Database. Incluye el esquema de las 25 tablas, la justificación del uso de arcos, la verificación de la primera, segunda y tercera forma normal y, al final, el listado de dependencias funcionales (DF) por tabla. El diagrama relacional se entrega como PDF independiente (diagrama_relacional.pdf).

Transformación MER → relacional:

- Cada entidad fuerte se convierte en una tabla con su identificador como llave primaria.
- Las relaciones 1:N se implementan con una llave foránea en la tabla del lado N.
- Las relaciones N:M sin atributos (EMPLOYEE–ROLE y PROMOTION–VARIANT) se resuelven con una tabla intermedia cuya llave es la combinación de las dos llaves foráneas.
- Las entidades asociativas con atributos propios (SALE_DETAIL, LAYAWAY_DETAIL, RETURN_DETAIL, BATCH_DETAIL) se convierten en tablas con llave compuesta.
- La jerarquía PERSON → CUSTOMER / EMPLOYEE se implementa con una tabla por supertipo y una por subtipo; la llave del subtipo es llave foránea hacia PERSON.
- La relación 1:1 LAYAWAY–SALE se implementa con una llave foránea única en SALE.

2. Esquema relacional

Además de las llaves primarias y las 35 llaves foráneas, el modelo declara restricciones UNIQUE sobre las llaves candidatas, restricciones CHECK de dominio (tallas, estados, tipos de resolución y de movimiento) y de valores positivos en montos y cantidades, y arcos exclusivos en PAYMENT (venta o apartado) y STORE_CREDIT_MOVEMENT (devolución o pago).

Personas y roles

PERSON (<u>id</u>, first_name, last_name, phone**)**

CUSTOMER (<u>id</u>, is_active**)**

FK {id} → PERSON(id)

EMPLOYEE (<u>id</u>, hire_date, is_active**)**

FK {id} → PERSON(id)

ROLE (<u>role_id</u>, name**)**

UK {name}

EMPLOYEE_ROLE (<u>EMPLOYEE_id</u>, **<u>ROLE_role_id</u>)**

FK {EMPLOYEE_id} → EMPLOYEE(id); FK {ROLE_role_id} → ROLE(role_id)

Catálogo

CATEGORY (category_id, name**)**

UK {name}

BRAND (brand_id, brand_name**)**

UK {brand_name}

PRODUCT (product_id, sku, name, is_active, updated_at, CATEGORY_category_id, BRAND_brand_id, EMPLOYEE_id**)**

UK {sku}; FK {BRAND_brand_id} → BRAND(brand_id); FK {EMPLOYEE_id} → EMPLOYEE(id); FK {CATEGORY_category_id} → CATEGORY(category_id)

VARIANT (variant_id, variant_size, color, material, price, stock, min_stock_threshold, PRODUCT_product_id**)**

UK {PRODUCT_product_id, variant_size, color, material}; FK {PRODUCT_product_id} → PRODUCT(product_id)

PROMOTION (promotion_id, fixed_price, start_date, end_date**)**

PROMOTION_VARIANT (VARIANT_variant_id, **PROMOTION_promotion_id)**

FK {PROMOTION_promotion_id} → PROMOTION(promotion_id); FK {VARIANT_variant_id} → VARIANT(variant_id)

Inventario y proveedores

SUPPLIER (id, name, phone, building, local_number, is_active**)**

BATCH (id, entry_date, SUPPLIER_id**)**

FK {SUPPLIER_id} → SUPPLIER(id)

BATCH_DETAIL (quantity, unit_cost, **BATCH_id**, **VARIANT_variant_id)**

FK {VARIANT_variant_id} → VARIANT(variant_id); FK {BATCH_id} → BATCH(id)

INVENTORY_ADJUSTMENT (id, quantity_change, reason, adjustment_date, VARIANT_variant_id, EMPLOYEE_id**)**

FK {VARIANT_variant_id} → VARIANT(variant_id); FK {EMPLOYEE_id} → EMPLOYEE(id)

Ventas, apartados y pagos

SALE (sale_id, sale_date, total, CUSTOMER_id, EMPLOYEE_id, LAYAWAY_layaway_id**)**

UK {LAYAWAY_layaway_id}; FK {EMPLOYEE_id} → EMPLOYEE(id); FK {LAYAWAY_layaway_id} → LAYAWAY(layaway_id); FK {CUSTOMER_id} → CUSTOMER(id)

SALE_DETAIL (quantity, applied_price, **SALE_sale_id**, **VARIANT_variant_id)**

FK {SALE_sale_id} → SALE(sale_id); FK {VARIANT_variant_id} → VARIANT(variant_id)

LAYAWAY (layaway_id, created_date, due_date, status, CUSTOMER_id, EMPLOYEE_id**)**

FK {EMPLOYEE_id} → EMPLOYEE(id); FK {CUSTOMER_id} → CUSTOMER(id)

LAYAWAY_DETAIL (quantity, unit_price, **LAYAWAY_layaway_id**, **VARIANT_variant_id)**

FK {VARIANT_variant_id} → VARIANT(variant_id); FK {LAYAWAY_layaway_id} → LAYAWAY(layaway_id)

PAYMENT_METHOD (id, name**)**

UK {name}

PAYMENT (payment_id, amount, payment_date, PAYMENT_METHOD_id, SALE_sale_id, LAYAWAY_layaway_id**)**

FK {PAYMENT_METHOD_id} → PAYMENT_METHOD(id); FK {SALE_sale_id} → SALE(sale_id); FK {LAYAWAY_layaway_id} → LAYAWAY(layaway_id)

Devoluciones, crédito y encargos

RETURN (<u>return_id</u>, return_date, resolution_type, SALE_sale_id**)**

FK {SALE_sale_id} → SALE(sale_id)

RETURN_DETAIL (quantity, **<u>RETURN_return_id</u>**, **<u>VARIANT_variant_id</u>**)**

FK {VARIANT_variant_id} → VARIANT(variant_id); FK {RETURN_return_id} → RETURN(return_id)

STORE_CREDIT_MOVEMENT (<u>movement_id</u>, movement_type, amount, movement_date, CUSTOMER_id, RETURN_return_id, PAYMENT_payment_id**)**

UK {PAYMENT_payment_id}; UK {RETURN_return_id}; FK {CUSTOMER_id} → CUSTOMER(id); FK {RETURN_return_id} → RETURN(return_id); FK {PAYMENT_payment_id} → PAYMENT(payment_id)

SPECIAL_ORDER (<u>special_order_id</u>, description, order_date, deposit_amount, status, origin_city, CUSTOMER_id, EMPLOYEE_id**)**

FK {EMPLOYEE_id} → EMPLOYEE(id); FK {CUSTOMER_id} → CUSTOMER(id)

3. Uso de arcos exclusivos en lugar de tablas adicionales

El modelo usa arcos exclusivos en tres puntos. Un arco agrupa dos relaciones de una misma tabla y exige que cada fila participe exactamente en una de ellas. En el relacional se traduce en dos llaves foráneas que admiten nulos y una restricción CHECK que obliga a que exactamente una tenga valor.

PAYMENT: venta o apartado

Un pago pertenece a una venta o a un apartado, nunca a ambos. Las alternativas eran:

- Dos tablas, SALE_PAYMENT y LAYAWAY_PAYMENT. Duplican la misma estructura (monto, fecha, método). Además, cualquier consulta sobre todos los pagos, como el total recaudado por método de pago o los pagos con crédito de tienda, necesitaría un UNION. STORE_CREDIT_MOVEMENT tendría que referenciar dos tablas distintas.
- Una sola columna genérica (transaction_id + tipo). No permite declarar una llave foránea real, así que la base de datos no garantiza que el pago apunte a una venta o a un apartado que exista.
- Una supertabla TRANSACTION de la que hereden SALE y LAYAWAY. Agrega una tabla y un join en cada consulta sin aportar atributos nuevos.

Con el arco hay una sola tabla de pagos, cada FK conserva su integridad referencial y el CHECK impide pagos sin transacción o con las dos. El costo es una columna nula por fila, que es despreciable.

STORE_CREDIT_MOVEMENT: devolución o pago

Cada movimiento de crédito nace de una devolución (ABONO) o de un pago que usa el crédito (REDENCION). Separarlo en dos tablas obligaría a sumar ambas para conocer el saldo del cliente, que es la consulta principal de HU-13. Con el arco, el saldo es una sola suma sobre una sola tabla. Las restricciones UNIQUE sobre RETURN_return_id y PAYMENT_payment_id garantizan que una devolución o un pago generen como máximo un movimiento.

PERSON → CUSTOMER / EMPLOYEE: subtipos exclusivos

La jerarquía se implementó con una tabla por subtipo y un arco que indica que una persona es cliente o empleado. Una sola tabla PERSON con un campo de tipo dejaría hire_date nulo en todos los clientes, y además las llaves foráneas de SALE, LAYAWAY y SPECIAL_ORDER no podrían distinguir entre ambos roles. Con tablas separadas, SALE.EMPLOYEE_id solo acepta empleados y SALE.CUSTOMER_id solo acepta clientes, de modo que el propio esquema garantiza que toda venta tenga un empleado y un cliente válidos (RN-04).

4. Verificación de la normalización

Primera forma normal (1FN)

- Todos los atributos son atómicos: el nombre de la persona se separa en first_name y last_name, y cada fila guarda un solo teléfono.
- No existen grupos repetidos: las listas (productos de una venta, roles de un empleado, variantes de una promoción, pagos de una venta) se llevaron a tablas propias.
- Todas las tablas tienen llave primaria.

Segunda forma normal (2FN)

Solo las tablas con llave compuesta pueden tener dependencias parciales. En todas ellas los atributos no primos dependen de la llave completa:

- SALE_DETAIL {SALE_sale_id, VARIANT_variant_id} → quantity, applied_price: la cantidad y el precio dependen de la línea concreta, no solo de la venta ni solo de la variante.
- LAYAWAY_DETAIL {LAYAWAY_layaway_id, VARIANT_variant_id} → quantity, unit_price.
- BATCH_DETAIL {BATCH_id, VARIANT_variant_id} → quantity, unit_cost: el costo unitario cambia de un lote a otro.
- RETURN_DETAIL {RETURN_return_id, VARIANT_variant_id} → quantity.
- EMPLOYEE_ROLE y PROMOTION_VARIANT no tienen atributos no primos.

Tercera forma normal (3FN)

No hay dependencias transitivas de atributos no primos. Los casos que podían parecerlo se resolvieron así:

- SALE: el cliente de una venta generada desde un apartado coincide con el del apartado. Como LAYAWAY_layaway_id es UNIQUE en SALE, es llave candidata y la dependencia parte de una superllave.
- STORE_CREDIT_MOVEMENT: RETURN_return_id y PAYMENT_payment_id son UNIQUE (un abono por devolución y una redención por pago), así que también son llaves candidatas.
- Catálogos (CATEGORY, BRAND, ROLE, PAYMENT_METHOD): el nombre es UNIQUE y es llave candidata alterna.
- VARIANT: {PRODUCT_product_id, variant_size, color, material} es llave candidata (no se repite una variante) y el precio no depende funcionalmente del material.

Atributos derivados (redundancia controlada)

SALE.total y VARIANT.stock no violan la 3FN, porque no dependen de otros atributos de su misma fila sino de filas de otras tablas (SALE_DETAIL; BATCH_DETAIL, SALE_DETAIL, LAYAWAY_DETAIL, RETURN_DETAIL e INVENTORY_ADJUSTMENT). Se conservan deliberadamente para que la validación de RN-02 (no vender más del stock disponible) y los reportes de ventas sean eficientes; su consistencia se garantiza en la implementación de la base de datos.

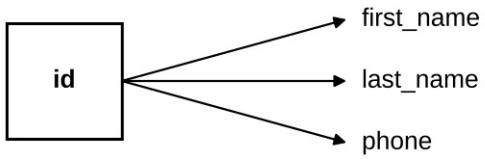
5. Listado de dependencias funcionales

En todas las tablas el único determinante de los atributos no primos es una llave candidata, por lo que el modelo cumple la 3FN (y la FNBC).

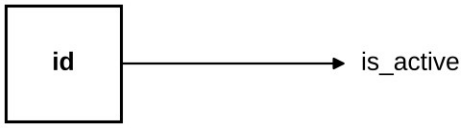
Personas y roles

PERSON	**Supertipo con los datos comunes de clientes y empleados.**

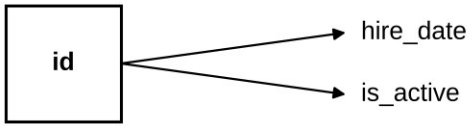
Llaves candidatas	{id}
Dependencias funcionales	DF1: {id} → first_name, last_name, phone
Forma normal	3FN



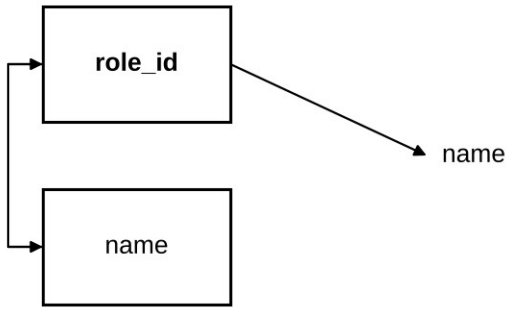
CUSTOMER	**Subtipo de PERSON (tabla por subtipo). Su llave es a la vez llave foránea hacia PERSON.**
Llaves candidatas	{id}
Dependencias funcionales	DF1: {id} → is_active
Forma normal	3FN — Herencia: un arco sobre CUSTOMER y EMPLOYEE hace excluyentes los subtipos.



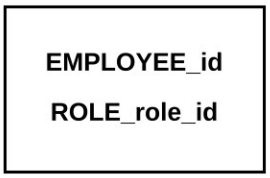
EMPLOYEE	**Subtipo de PERSON. Guarda la fecha de vinculación y el estado (RN-04).**
Llaves candidatas	{id}
Dependencias funcionales	DF1: {id} → hire_date, is_active
Forma normal	3FN



ROLE	**Catálogo de roles (cajero, bodeguero).**
Llaves candidatas	{role_id}, {name}
Dependencias funcionales	DF1: {role_id} → name DF2: {name} → role_id
Forma normal	3FN



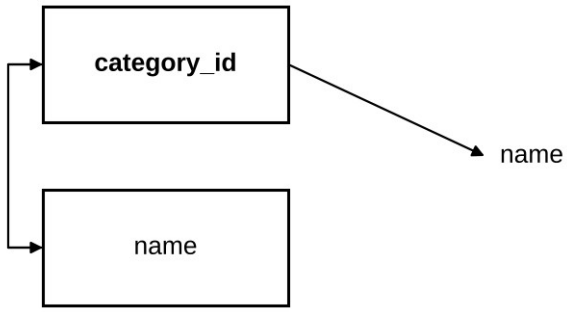
EMPLOYEE_ROLE	**Resuelve la relación N:M EMPLOYEE–ROLE del MER (roles simultáneos, HU-06).**
Llaves candidatas	{EMPLOYEE_id, ROLE_role_id}
Dependencias funcionales	DF1: Solo dependencias triviales (la tabla está formada únicamente por su llave).
Forma normal	3FN



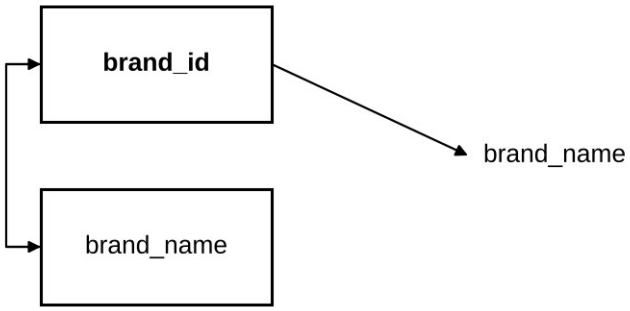
Solo dependencias triviales

Catálogo

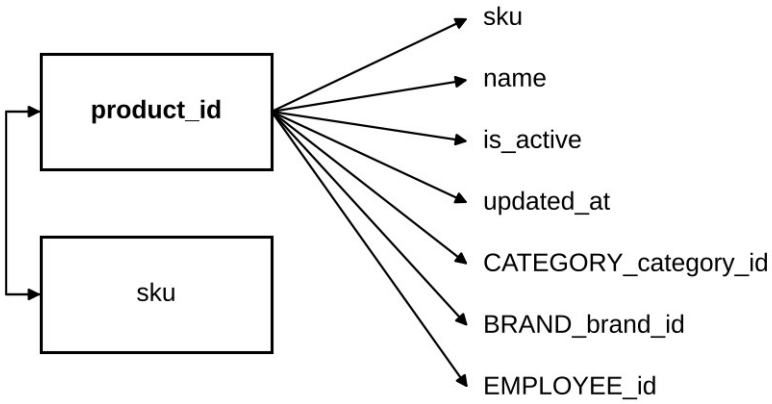
CATEGORY	**Catálogo de categorías de prenda.**
Llaves candidatas	{category_id}, {name}
Dependencias funcionales	DF1: {category_id} → name DF2: {name} → category_id
Forma normal	3FN



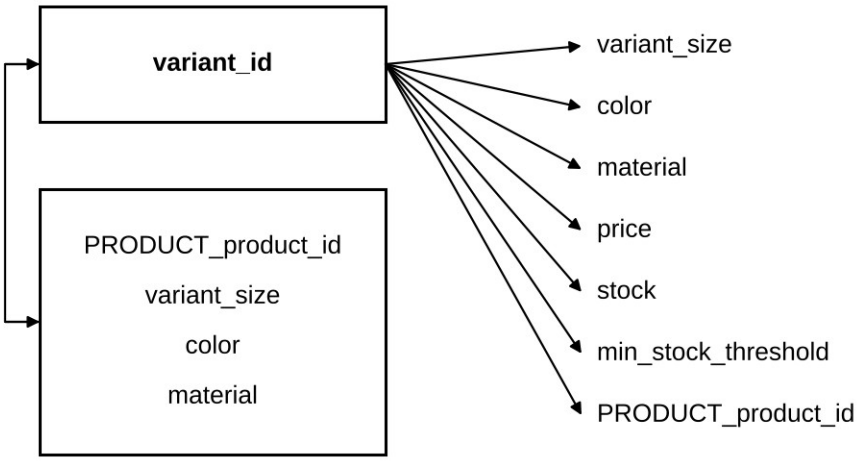
BRAND	**Catálogo de marcas.**
Llaves candidatas	{brand_id}, {brand_name}
Dependencias funcionales	DF1: {brand_id} → brand_name DF2: {brand_name} → brand_id
Forma normal	3FN



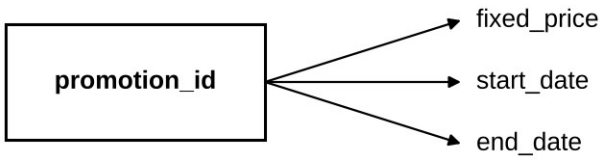
PRODUCT	**Diseño de prenda con SKU único (RN-01, HU-01).**
Llaves candidatas	{product_id}, {sku}
Dependencias funcionales	DF1: {product_id} → sku, name, is_active, updated_at, CATEGORY_category_id, BRAND_brand_id, EMPLOYEE_id DF2: {sku} → product_id, name, is_active, updated_at, CATEGORY_category_id, BRAND_brand_id, EMPLOYEE_id
Forma normal	3FN



VARIANT	**Prenda física vendible: talla, color y material de un producto (RN-01, HU-02).**
Llaves candidatas	{variant_id}, {PRODUCT_product_id, variant_size, color, material}
Dependencias funcionales	DF1: {variant_id} → variant_size, color, material, price, stock, min_stock_threshold, PRODUCT_product_id DF2: {PRODUCT_product_id, variant_size, color, material} → variant_id, price, stock, min_stock_threshold
Forma normal	3FN — variant_size, color, material y price no determinan a otros atributos: el caso fija el precio por tipo de tela solo para la tela gruesa ($90.000) y da un rango para la tela fría, por lo que material →/ price. stock es un atributo derivado (ver sección 4).



PROMOTION	**Precio fijo con vigencia (HU-14).**
Llaves candidatas	{promotion_id}
Dependencias funcionales	DF1: {promotion_id} → fixed_price, start_date, end_date
Forma normal	3FN



PROMOTION_VARIANT	**Resuelve la relación N:M PROMOTION–VARIANT del MER.**
Llaves candidatas	{VARIANT_variant_id, PROMOTION_promotion_id}
Dependencias funcionales	DF1: Solo dependencias triviales (la tabla está formada únicamente por su llave).
Forma normal	3FN

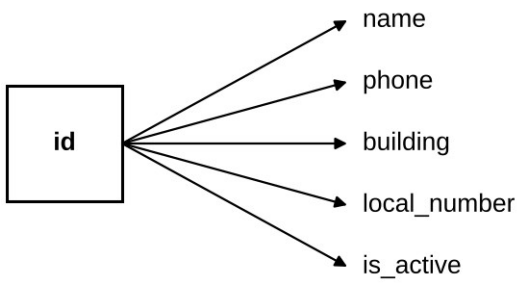


Solo dependencias triviales

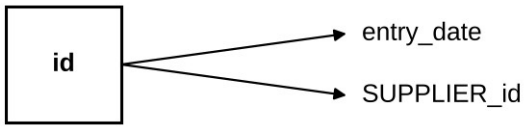
Inventario y proveedores

SUPPLIER	**Proveedor presencial con teléfono, edificio y local (RN-05).**
Llaves candidatas	{id}
Dependencias funcionales	DF1: {id} → name, phone, building, local_number,

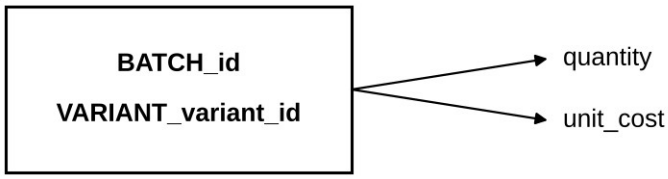
	is_active
Forma normal	3FN



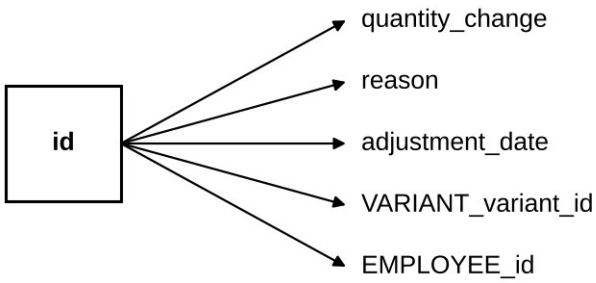
BATCH	**Lote de mercancía recibido de un proveedor (HU-04).**
Llaves candidatas	{id}
Dependencias funcionales	DF1: {id} → entry_date, SUPPLIER_id
Forma normal	3FN



BATCH_DETAIL	**Entidad asociativa lote–variante con cantidad y costo unitario.**
Llaves candidatas	{BATCH_id, VARIANT_variant_id}
Dependencias funcionales	DF1: {BATCH_id, VARIANT_variant_id} → quantity, unit_cost
Forma normal	3FN

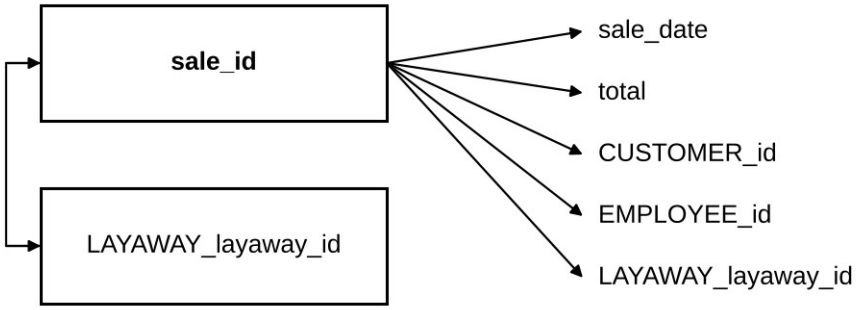


INVENTORY_ADJUSTMENT	**Corrección de inventario registrada por un empleado (HU-04).**
Llaves candidatas	{id}
Dependencias funcionales	DF1: {id} → quantity_change, reason, adjustment_date, VARIANT_variant_id, EMPLOYEE_id
Forma normal	3FN

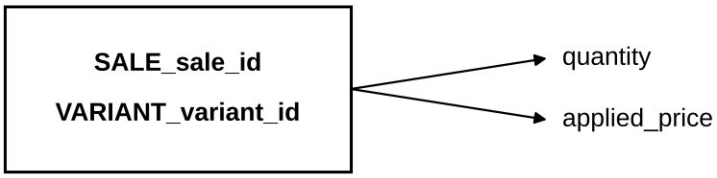


Ventas, apartados y pagos

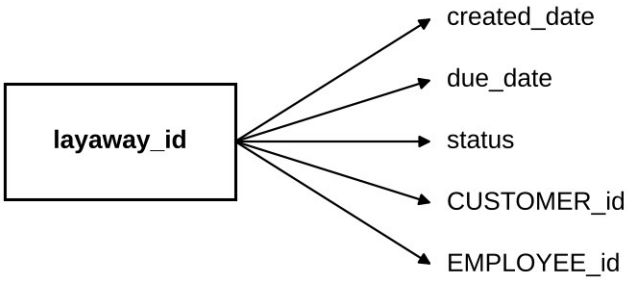
SALE	**Venta en tienda física (HU-07). Puede originarse en un apartado.**
Llaves candidatas	{sale_id}, {LAYAWAY_layaway_id}
Dependencias funcionales	DF1: {sale_id} → sale_date, total, CUSTOMER_id, EMPLOYEE_id, LAYAWAY_layaway_id DF2: {LAYAWAY_layaway_id} → sale_id, sale_date, total, CUSTOMER_id, EMPLOYEE_id
Forma normal	3FN — LAYAWAY_layaway_id es UNIQUE: es llave candidata, por lo que LAYAWAY_layaway_id → CUSTOMER_id no viola la 3FN. total es un atributo derivado (ver sección 4).



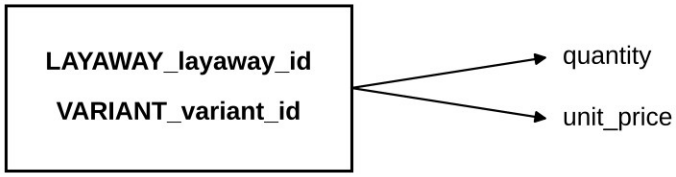
SALE_DETAIL	**Entidad asociativa venta–variante con cantidad y precio aplicado.**
Llaves candidatas	{SALE_sale_id, VARIANT_variant_id}
Dependencias funcionales	DF1: {SALE_sale_id, VARIANT_variant_id} → quantity, applied_price
Forma normal	3FN — applied_price es el precio histórico efectivamente cobrado (puede venir de una promoción); no depende de VARIANT_variant_id solo, sino de la línea de venta completa.



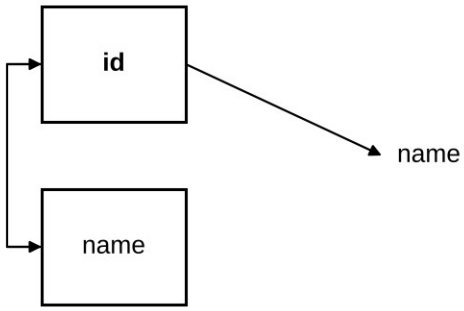
LAYAWAY	Apartado con fecha límite y estado (HU-09).
Llaves candidatas	{layaway_id}
Dependencias funcionales	DF1: {layaway_id} → created_date, due_date, status, CUSTOMER_id, EMPLOYEE_id
Forma normal	3FN



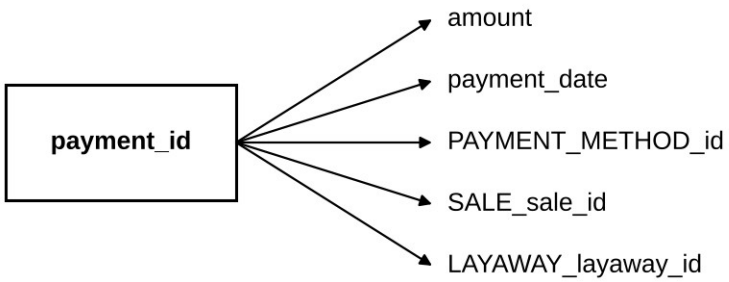
LAYAWAY_DETAIL	Entidad asociativa apartado–variante con cantidad y precio pactado.
Llaves candidatas	{LAYAWAY_layaway_id, VARIANT_variant_id}
Dependencias funcionales	DF1: {LAYAWAY_layaway_id, VARIANT_variant_id} → quantity, unit_price
Forma normal	3FN — unit_price es el precio pactado al apartar; depende de la línea completa (apartado, variante).



PAYMENT_METHOD	Catálogo: efectivo, transferencia, Sistecrédito, crédito de tienda.
Llaves candidatas	{id}, {name}
Dependencias funcionales	DF1: {id} → name DF2: {name} → id
Forma normal	3FN

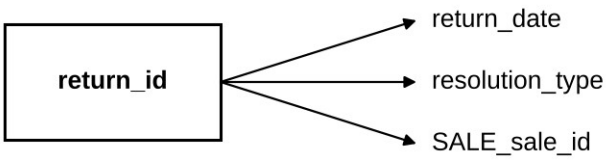


PAYMENT	**Pago o abono de una venta o de un apartado; admite pagos mixtos (HU-08, HU-10).**
Llaves candidatas	{payment_id}
Dependencias funcionales	DF1: {payment_id} → amount, payment_date, PAYMENT_METHOD_id, SALE_sale_id, LAYAWAY_layaway_id
Forma normal	3FN — El arco PAYMENT_ARC obliga a que el pago pertenezca exactamente a una venta o a un apartado.



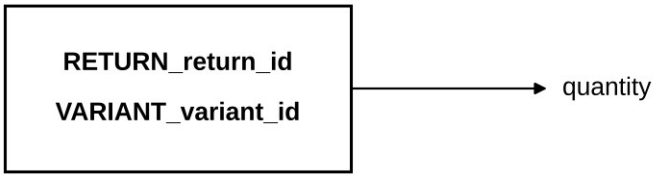
Devoluciones, crédito y encargos

RETURN	**Devolución de una venta, resuelta con crédito o cambio (HU-12).**
Llaves candidatas	{return_id}
Dependencias funcionales	DF1: {return_id} → return_date, resolution_type, SALE_sale_id
Forma normal	3FN

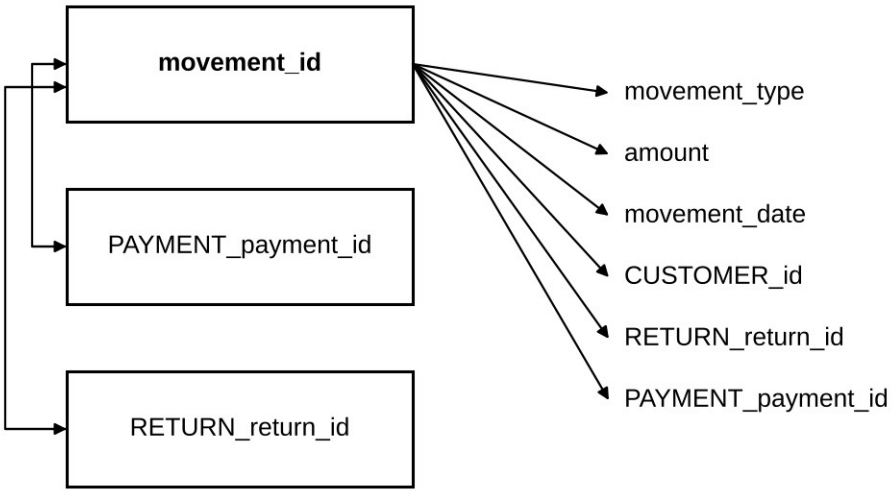


RETURN_DETAIL	**Entidad asociativa devolución–variante con la**

	cantidad devuelta.
Llaves candidatas	{RETURN_return_id, VARIANT_variant_id}
Dependencias funcionales	DF1: {RETURN_return_id, VARIANT_variant_id} → quantity
Forma normal	3FN



STORE_CREDIT_MOVEMENT	**Historial de crédito de tienda: abonos por devolución y redenciones en pagos (HU-13).**
Llaves candidatas	{movement_id}, {PAYMENT_payment_id}, {RETURN_return_id}
Dependencias funcionales	DF1: {movement_id} → movement_type, amount, movement_date, CUSTOMER_id, RETURN_return_id, PAYMENT_payment_id DF2: {PAYMENT_payment_id} → movement_id, movement_type, amount, movement_date, CUSTOMER_id, RETURN_return_id DF3: {RETURN_return_id} → movement_id, movement_type, amount, movement_date, CUSTOMER_id, PAYMENT_payment_id
Forma normal	3FN — RETURN_return_id y PAYMENT_payment_id son UNIQUE (una devolución genera un solo abono y un pago una sola redención): son llaves candidatas, por lo que sus dependencias hacia CUSTOMER_id parten de una superllave. El arco obliga a que exactamente una de las dos tenga valor.



SPECIAL_ORDER	**Encargo de una prenda conseguida en otra ciudad (HU-11).**
Llaves candidatas	{special_order_id}
Dependencias funcionales	DF1: {special_order_id} → description, order_date,

	deposit_amount, status, origin_city, CUSTOMER_id, EMPLOYEE_id
Forma normal	3FN

